

Byte Pair Encoding (BPE) FULL CALCULATION (NO STEP MISSING)

Training text:

low low lower newest wide

We will process **exactly** like the original BPE paper (Sennrich et al., 2016).

STEP 1 — Preprocess (split words into characters +)

Add an end-of-word symbol:

low → low </w>

low → low </w>

lower → lower </w>

newest → newest </w>

wide → wide </w>

Full corpus:

1. low </w>
2. low </w>
3. lower </w>
4. newest </w>
5. wide </w>

STEP 2 — Count all adjacent pairs (FIRST ITERATION)

We extract all pairs inside each word.

Word 1: low

Pairs:

(l,o), (o,w), (w,</w>)

Word 2: low (same)

(l,o), (o,w), (w,</w>)

Word 3: lower

Pairs:

(l,o), (o,w), (w,e), (e,r), (r,</w>)

Word 4: newest

Pairs:

(n,e), (e,w), (w,e), (e,s), (s,t), (t,</w>)

Word 5: wide

Pairs:

(w,i), (i,d), (d,e), (e,</w>)

STEP 3 — Count pair frequencies

Pair	Count
(l,o)	3
(o,w)	3
(w,))	2
(w,e)	2
(e,w)	1
(e,r)	1
(r,))	1
(n,e)	1
(e,s)	1
(s,t)	1
(t,))	1
(w,i)	1
(i,d)	1
(d,e)	1
(e,))	1

STEP 4 — Choose the most frequent pair

Most frequent pairs:

(l,o) = 3

(o,w) = 3

Tie → choose (l,o).

STEP 5 — MERGE #1 → (l,o) → "lo"

Update all words:

1. lo w </w>
2. lo w </w>
3. lo w e r </w>
4. n e w e s t </w>
5. w i d e </w>

STEP 6 — RECALCULATE pairs (SECOND ITERATION)

We repeat Steps 2–3.

New pairs count:

Word 1: lo w

(lo,w), (w,</w>)

Word 2: lo w

(lo,w), (w,</w>)

Word 3: lo w e r

(lo,w), (w,e), (e,r), (r,</w>)

Word 4: n e w e s t

(same as before)

(n,e), (e,w), (w,e), (e,s), (s,t), (t,</w>)

Word 5: w i d e

(w,i), (i,d), (d,e), (e,</w>)

Pair frequencies now

Pair	Count
(lo,w)	3
(w,)	2
(w,e)	2
all others	1

STEP 7 — MERGE #2 → (lo,w) → “low”

Update all words:

1. low </w>

2. low </w>
3. low e r </w>
4. n e w e s t </w>
5. w i d e </w>

STEP 8 — THIRD ITERATION: Count pairs

Word 1: low

(low,</w>)

Word 2: (same)

(low,</w>)

Word 3:

(low,e), (e,r), (r,</w>)

Word 4:

(n,e), (e,w), (w,e), (e,s), (s,t), (t,</w>)

Word 5:

(w,i), (i,d), (d,e), (e,</w>)

Pair counts

Pair	Count
(low,)	2
others	1

STEP 9 — MERGE #3 → (low,) → “low”

Update corpus:

1. low</w>
2. low</w>
3. low e r </w>
4. n e w e s t </w>
5. w i d e </w>

STEP 10 — FOURTH ITERATION: Count pairs

Word 3:

(low,e), (e,r), (r,</w>)

Word 4:

(n,e), (e,w), (w,e), (e,s), (s,t), (t,</w>)

Word 5:

(w,i), (i,d), (d,e), (e,</w>)

Pair counts

All pairs appear once:

Pair	Count
------	-------

(low,e)	1
---------	---

(e,r)	1
-------	---

(r,)	1
------	---

(n,e)	1
-------	---

(e,w)	1
-------	---

(w,e)	1
-------	---

(e,s)	1
-------	---

(s,t)	1
-------	---

(t,)	1
------	---

(w,i)	1
-------	---

(i,d)	1
-------	---

(d,e)	1
-------	---

(e,)	1
------	---

Choose the first by order:

→ (e,r)

STEP 11 — MERGE #4 → (e,r) → "er"

Update:

low</w>

low</w>

low er </w>

new est </w>

wide </w>

STEP 12 — FIFTH ITERATION: Count pairs

Pairs with count 1 again.

Next by order → (e,w)

STEP 13 — MERGE #5 → (e,w) → "ew"

Update:

low</w>

low</w>

low er </w>

new est </w>

wide </w>

STEP 14 — SIXTH ITERATION

Next merge (first in order) → (d,e)

STEP 15 — MERGE #6 → (d,e) → "de"

Update:

low</w>

low</w>

low er </w>

new est </w>

wide </w>

STEP 16 — SEVENTH ITERATION

Next merge → (i,d)

STEP 17 — MERGE #7 → (i,d) → "id"

Update:

low</w>

low</w>

low er </w>

new est </w>

wid e </w>

FINAL — BPE VOCABULARY

Characters:

l, o, w, e, r, n, s, t, i, d, </w>

Merged tokens:

lo

low

low</w>

er

ew

de

id

FINAL BPE VOCAB

l

o

w

e

r

n

s

t

i

d

</w>

lo

low

low</w>

er

ew

de

id

WORDPIECE TOKENIZER — COMPLETE STEP-BY-STEP TUTORIAL

(Using your example: low low lower newest wide)

WordPiece is the subword algorithm used by **BERT**, **ALBERT**, **DistilBERT**, etc.

It is similar to BPE, but the **merge rule is different**.

1. What is WordPiece?

WordPiece is a subword tokenization algorithm that:

- Starts from characters
- Merges pairs based on a scoring function
- Produces tokens like:
- low, ##er, n, ##ew, ##est

WordPiece solves a key NLP problem:

It avoids big vocabularies while preventing weird merges on rare symbols.

2. WordPiece Scoring Formula

BPE = picks pair with highest frequency.

WordPiece = picks pair that **maximizes the probability of generating training text**.

Practically, WordPiece uses:

$$\text{score}(a, b) = \frac{\text{freq}(a, b)}{\text{freq}(a)}$$

Where:

- **freq(a)** = how many times token *a* appears
- **freq(a,b)** = how many times *pair* (a,b) appears
- **a** = left symbol
- **b** = right symbol

This is why WordPiece is more stable.

3. Training Text

We use your text:

low low lower newest wide

4. Step 1 — Split Words into Characters

WordPiece does NOT add $\langle w \rangle$ like BPE.

low $\rightarrow$ l o w

low $\rightarrow$ l o w

lower $\rightarrow$ l o w e r

newest $\rightarrow$ n e w e s t

wide $\rightarrow$ w i d e

5. Step 2 — Count Single-Character Frequencies

Count how many times each character appears in the entire dataset:

Character	Count
l	3
o	3
w	5
e	5
r	1
n	1
s	1
t	1
i	1
d	1

These counts will be used in the denominator.

6. Step 3 — Count All Adjacent Pairs

From each word:

From "l o w":

(l,o)

(o,w)

From "l o w e r":

(l,o), (o,w), (w,e), (e,r)

From "n e w e s t":

(n,e), (e,w), (w,e), (e,s), (s,t)

From "w i d e":

(w,i), (i,d), (d,e)

Now make the frequency table:

Pair	Count
(l,o)	3
(o,w)	3
(w,e)	2
(e,w)	1
(e,r)	1
(n,e)	1
(e,s)	1
(s,t)	1
(w,i)	1
(i,d)	1
(d,e)	1

7. Step 4 — Compute WordPiece Scores

Use:

$$\frac{\text{freq}(a,b)}{\text{freq}(a)}$$

(l,o)

- $\text{freq}(l,o) = 3$
- $\text{freq}(l) = 3$
- $\text{score} = 3/3 = 1.0$

(o,w)

- $\text{freq}(o,w) = 3$
- $\text{freq}(o) = 3$
- $\text{score} = 1.0$

(w,e)

- $\text{freq}(w,e) = 2$
- $\text{freq}(w) = 5$
- $\text{score} = 2/5 = 0.4$

All others < 0.4

So highest scores:

(l,o) = 1.0

(o,w) = 1.0

Tie $\rightarrow$ choose lexicographically $\rightarrow$ (l,o)

MERGE #1 $\rightarrow$ (l, o) $\rightarrow$ "lo"

Update words:

lo w

lo w

lo w e r

n e w e s t

w i d e

8. Recount Pairs After Merge #1

Now pairs:

"lo w"

(lo, w)

"lo w e r"

(lo,w), (w,e), (e,r)

"n e w e s t"

Same as before.

Make the table:

Pair	Count
(lo,w)	3
(w,e)	2
others	1

Now compute scores:

(lo,w)

$\text{freq}(lo,w)=3$

$\text{freq}(lo)=3$

$\text{score} = 1.0$

(w,e)

$2/5 = 0.4$

Highest:

(lo, w)

MERGE #2 $\rightarrow$ (lo, w) $\rightarrow$ "low"

Corpus becomes:

low

low

low e r

n e w e s t

w i d e

9. Recount Pairs After Merge #2

Pairs:

"low e r"

(low,e), (e,r)

"n e w e s t"

(n,e), (e,w), (w,e), (e,s), (s,t)

"w i d e"

(w,i), (i,d), (d,e)

Count table:

Pair	Count
(low,e)	1
(e,r)	1
(n,e)	1
(e,w)	1
(w,e)	1
(e,s)	1
(s,t)	1

(w,i) 1

(i,d) 1

(d,e) 1

Now compute scores:

Example:

(w,e)

$\text{freq}(w,e)=1$

$\text{freq}(w)=5$

$\text{score} = 1/5 = 0.2$

(e,r)

$\text{freq}(e,r)=1$

$\text{freq}(e)=5$

$\text{score} = 0.2$

(low,e)

$\text{freq}(\text{low},e)=1$

$\text{freq}(\text{low})=2$

$\text{score} = 0.5 \leftarrow \text{highest!}$

MERGE #3 $\rightarrow (\text{low}, e) \rightarrow \text{"lowe"}$

Updated:

low

low

lowe r

n e w e s t

w i d e

10. Repeat Scoring and Merging

Next merges based on score:

MERGE #4 $\rightarrow (w,e) \rightarrow \text{"we"}$

MERGE #5 $\rightarrow (e,r) \rightarrow \text{"er"}$

MERGE #6 $\rightarrow (d,e) \rightarrow \text{"de"}$

MERGE #7 $\rightarrow (i,d) \rightarrow \text{"id"}$

After all merges:

low

low

lowe r

n w e e s t

w i d e

11. Final WordPiece Vocabulary

Base characters:

l, o, w, e, r, n, s, t, i, d

Merged tokens:

lo

low

lowe

we

er

de

id

WordPiece vocabulary format (BERT):

- First subword: no prefix
- Continuation subwords: prefix with ##

Examples:

low $\rightarrow$ low

lower $\rightarrow$ low ##er

newest $\rightarrow$ n ##we ##e ##s ##t

wide $\rightarrow$ w ##id ##e

FINAL SUMMARY OF WORDPIECE

- WordPiece uses **probability-based scoring**, not raw frequency
- $\text{Score} = \frac{\text{pair}_{\text{freq}}}{\text{left}_{\text{token}}_{\text{freq}}}$
- Avoids merging rare tokens too early
- Produces stable and meaningful subwords
- Used by BERT, ALBERT, DistilBERT